

Théorie des langages

Olivier Ridoux

**grammaires algébriques
et
analyse syntaxique**

Plan

- Grammaires régulières
 - pour s'échauffer
- Grammaires algébriques
- Arbre de dérivation et ambiguïté
- Analyse syntaxique

Grammaires régulières

Principe des grammaires

- Représenter l'imbrication des catégories syntaxiques
(groupes syntagmatiques
☞ grammaires syntagmatiques)

Phrase → Sujet Verbe Complément

PhInterro → Que Verbe Sujet ?

il Verbe → il Verbe3gpe → il dort

Grammaire régulière

- Def. Vocabulaire **terminal** : Σ_T
 - l'alphabet ($\approx \Sigma$)
- Def. Vocabulaire **non terminal** : Σ_N
 - les **catégories syntaxiques**
- Def. **Axiome** $\in \Sigma_N$
 - souvent noté **S** (*start*) ou **X**
 - parfois positionnel

Grammaire régulière

- tête
- corps
- Def. **Règles** $\in \Sigma_N \rightarrow \Sigma_T \Sigma_N$
ou $\Sigma_N \rightarrow \varepsilon$

$$\underline{A} \rightarrow a B, A \rightarrow \varepsilon, B \rightarrow b A$$

Grammaires régulières

- Def. Dérivation immédiate

$$m \bullet nt \rightarrow m \bullet t \bullet nt'$$

ssi $nt \rightarrow t \ nt'$ est une règle

- Def. Dérivation $\rightarrow^+$
 - répéter dérivation immédiate

Grammaires régulières

- Def. Mot engendré

G engendre m ssi **Axiome $\rightarrow^+$ m**

- Def. Langage engendré

G engendre $\mathcal{L}$

ssi

$m \in \mathcal{L} \iff G \text{ engendre } m$

Grammaire régulière

- Th. GR engendre exactement les langages rationnels

➡ équivalent RE

➡ équivalent $\mathcal{M}$ (DFA, ...)

Grammaire régulière

- Variantes cosmétiques

(~ anodines ou bénignes)

– règles $\in \Sigma_N \rightarrow \Sigma_N \Sigma_T$ ou $\Sigma_N \rightarrow \varepsilon$

– règles $\in \Sigma_N \rightarrow \Sigma_N \Sigma_T$ ou $\Sigma_N \rightarrow \Sigma_T^{0|1}$

– règles $\in \Sigma_N \rightarrow \Sigma_N \Sigma_T^+$ ou $\Sigma_N \rightarrow \Sigma_T^*$

Grammaire régulière

- Pas de progrès / expressions régulières
- Mise en jambe pour faire mieux
- Retenir
 - vocabulaire **terminal, non terminal**
 - **axiome**
 - **règles, dérivation**
 - langage **engendré**

Grammaires algébriques

Grammaires algébriques

- Def. **Règles** $\in \Sigma_N \rightarrow (\Sigma_N \mid \Sigma_T)^*$

A $\rightarrow$ a **A** b

A $\rightarrow \epsilon$

D $\rightarrow$ d **' ; ' D**

D $\rightarrow$ d

C $\rightarrow$ 'while' **E 'do' Cs 'od'**

Cs $\rightarrow$ C **' ; ' Cs**

Cs $\rightarrow$ C

corps

tête



- Elles engendrent les structures **bien parenthésées**

S $\rightarrow$ T **S** , **S** $\rightarrow \epsilon$, **T** $\rightarrow$ '(' **S** ')'

langage de Dyck

Grammaires algébriques

- Def. Dérivation **immédiate**

$$m_1 \bullet nt \bullet m_2 \rightarrow m_1 \bullet m \bullet m_2$$

ssi $nt \rightarrow m$ est une règle

- Def. Dérivation $\rightarrow^+$
 - répéter dérivation immédiate

comparer avec
grammaires régulières

Grammaires algébriques

- Def. Mot engendré

G engendre m ssi **Axiome $\rightarrow^+$ m**

- Def. Langage engendré

G engendre $\mathcal{L}$

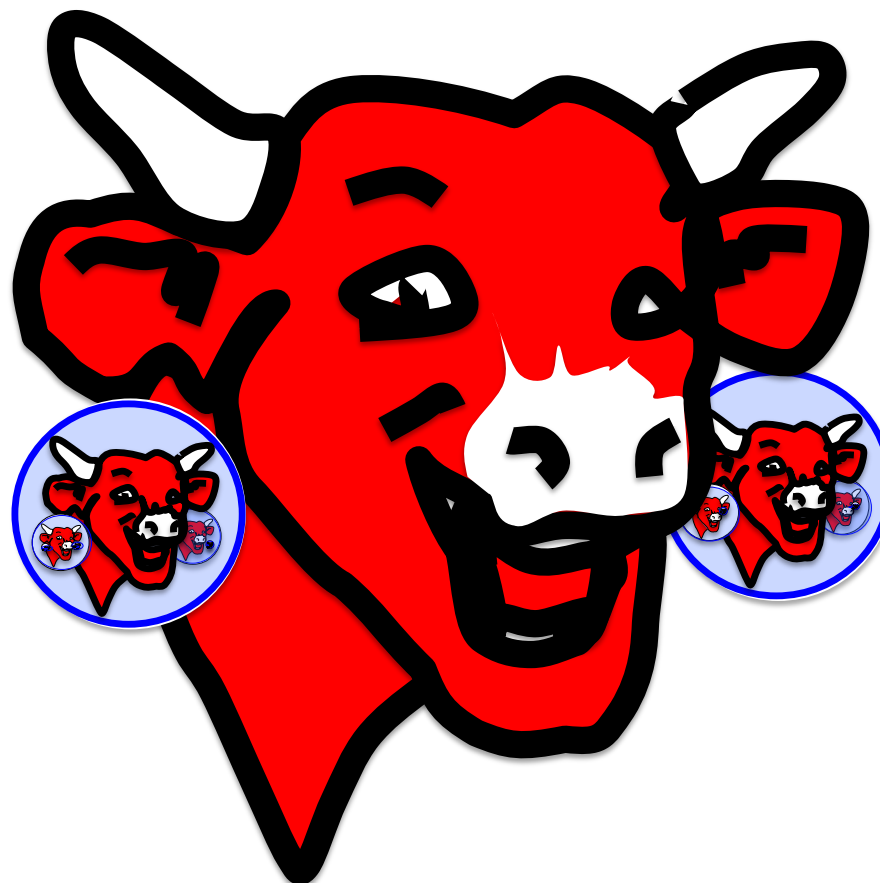
ssi

$m \in \mathcal{L} \iff G \text{ engendre } m$

Grammaires algébriques

- Def. Un **langage algébrique** est un langage engendré par une grammaire algébrique

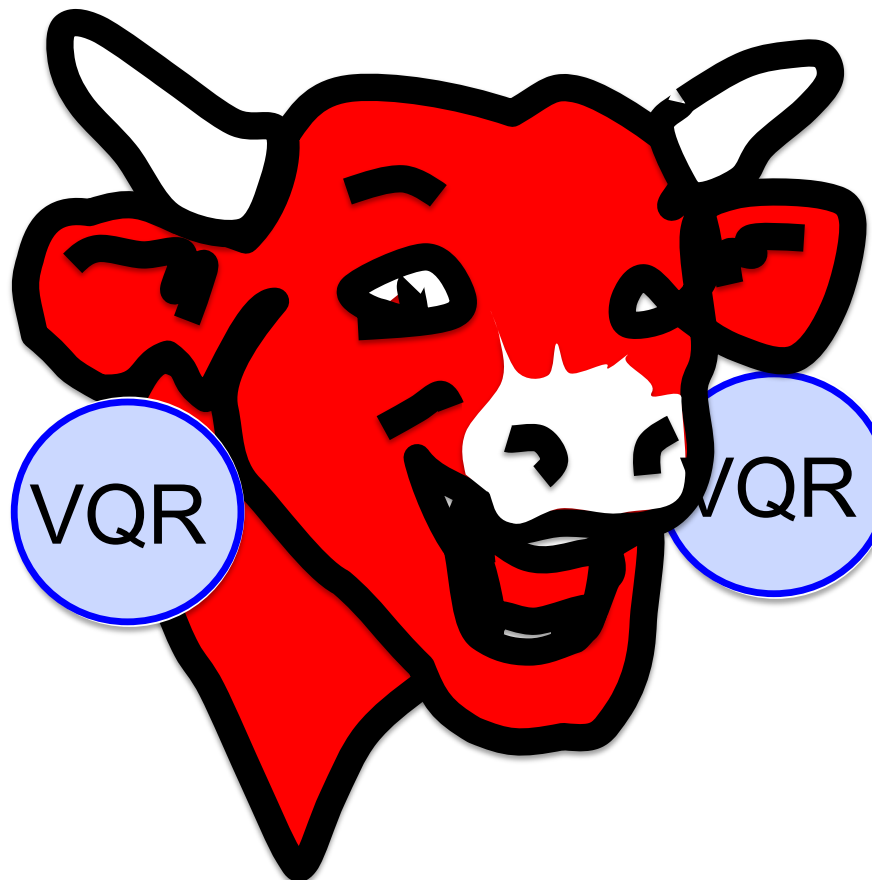
Le style RE



Le style GA

VQR $\rightarrow$

VQR $\rightarrow \varepsilon$



Grammaires algébriques

Th. **Les GA peuvent engendrer**

$a^n b^n$ pour $n \geq 0$

Th. **Les GA ne peuvent pas engendrer**

$a^n b^n c^n$ pour $n \geq 0$



- **Lemme d'itération** $\approx$ lemme de pompage pour grammaires algébriques

Grammaires algébriques

- Def. Variante **étendue** (mais cosmétique)

– règles $\in \Sigma_N \rightarrow \text{RE}(\Sigma_N \cup \Sigma_T)$

$D \rightarrow d \text{ ',' } D \quad D \rightarrow d$

$D \rightarrow (d \text{ ',' } D) \mid d$

$D \rightarrow (d \text{ ',' })^* d$

Grammaires algébriques

- Def. Forme normale de Chomsky
 - règles $\in \Sigma_N \rightarrow \Sigma_N \Sigma_N$ ou $\Sigma_N \rightarrow \Sigma_T$
 - facilite le raisonnement sur les GA
- Def. Forme normale de Greibach
 - règles $\in \Sigma_N \rightarrow \Sigma_T \Sigma_N^*$ ou $\Sigma_N \rightarrow \varepsilon$
 - facilite l'analyse syntaxique (à suivre)

comparer avec
grammaires régulières

Exemple : syntaxe du langage WHILE

- Syntaxe des programmes

Program $\rightarrow$ **Function** **<CR>** **Program** | **Function**

Function $\rightarrow$ 'function' **Symbol** ':' **Definition**

Definition $\rightarrow$ 'read' **Input**

'%' **Commands**

'%' 'write' **Output**

Input $\rightarrow$ **Variable** ',' **Input**

Input $\rightarrow$ **Variable**

Output $\rightarrow$ **Variable** ',' **Output** | **Variable**

<CR> =
Carriage-Return

Exemple : syntaxe de WHILE

- Syntaxe des commandes
 - Commands $\rightarrow$ Command ';' Commands**
 - Commands $\rightarrow$ Command**
 - Command $\rightarrow$ 'nop'**
 - Command $\rightarrow$ Vars ':=' Exprs**
 - Command $\rightarrow$ 'while' Expr 'do' Commands 'od'**
 - Vars $\rightarrow$ Variable ',' Vars | Variable**
 - Exprs $\rightarrow$ Expr ',' Exprs | Expr**

Exemple : syntaxe de WHILE

- Syntaxe des expressions

$\text{ExprBase} \rightarrow \text{'nil'} \mid \text{Variable} \mid \text{Symbol}$

$\text{ExprBase} \rightarrow \text{'(' 'cons' Lexpr ')'} \mid \text{'(' 'list' Lexpr ')')}$

$\text{ExprBase} \rightarrow \text{'(' 'hd' Expr ')'} \mid \text{'(' 'tl' Expr ')')}$

$\text{ExprBase} \rightarrow \text{'(' Symbol LExpr ')')}$

$\text{Expr} \rightarrow \text{ExprBase '=' ExprBase} \mid \text{ExprBase}$

$\text{LExpr} \rightarrow \text{Expr LExpr} \mid \varepsilon$

Exemple : syntaxe de WHILE

- Syntaxe des variables et symboles

Variable = Maj (Maj | Min | Dec)*[!|?]

Symbol = Min (Maj | Min | Dec)*[!|?]

- Séparateurs

SEP = <SP> | <CR> | <TAB> | <LF>

- on peut insérer des caractères de mise-en-page partout entre deux (non-)terminaux

BNF, EBNF et les autres

- Des notations conventionnelles
(Extended) Backus-Naur Form
- Convention de saisie sur clavier-écran (~ 1960)
 - écrire des grammaires **lisibles par des programmes**
 - **distinguer terminaux et non-terminaux**
 - **distinguer tête et corps**
 - (Extended) noter **alternative**, **fermeture** de Kleene, **options**, etc.

Plus souvent utilisé dans l'esprit que dans la lettre

EBNF en EBNF

syntax	::= { rule }	
rule	::= identifier ":: := " expression	
expression	::= term { " " term }	
term	::= factor { factor }	
factor	::= identifier	
	quoted_symbol	
	" (" expression ") "	structuration
	" [" expression "] "	option
	" { " expression " } "	répétition
identifier	::= letter { letter digit }	
quoted_symbol	::= " \" " { any_character } " \" "	

WHILE en EBNF

- Syntaxe des programmes

Program ::= {Function <CR>} Function

Function ::= "function" symbol ":" Definition

Definition ::= "read" Input "%" Commands "%" "write" Output

Input ::= { Var "," } Var

Output ::= { Var "," } Var

WHILE en EBNF

- Syntaxe des commandes

Commands ::= { Command ";" } Command

Command ::= "nop"

| Vars ":=" Expr

| "while" Expr "do" Commands "od"

| "if" Expr "then" Command ["else" Command] "fi"

Vars ::= { Var "," } Var

Exprs ::= { Expr "," } Expr

WHILE en EBNF

- Syntaxe des expressions

ExprBase ::= "nil" | Variable | Symbol
| "(" ("cons" | "list") {Expr} ")"
| "(" ("hd" | "tl") Expr ")"
| "(" Symbol {Expr} ")"

Expr ::= ExprBase | ExprBase "=?" ExprBase

Diagramme chemin de fer

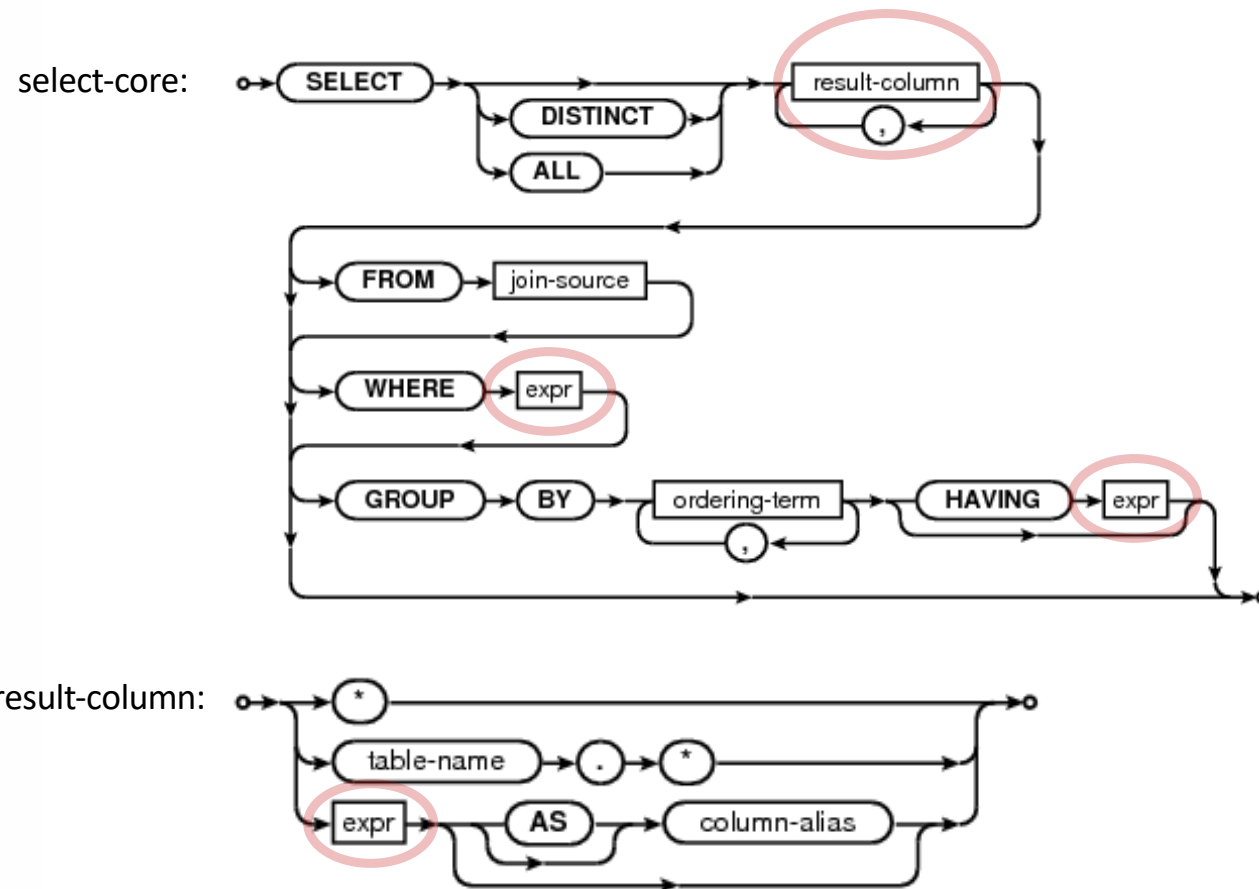
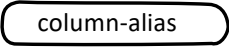
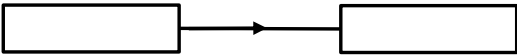
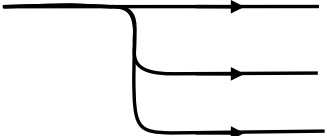
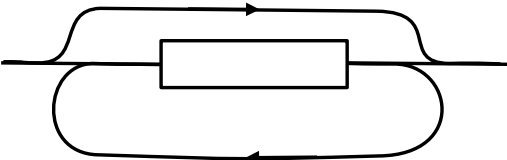


Diagramme chemin de fer

- Symbole non-terminal : 
- Symbole terminal : 
- Symbole terminal générique : 
- Produit : 
- Alternative : 
- Fermeture : 

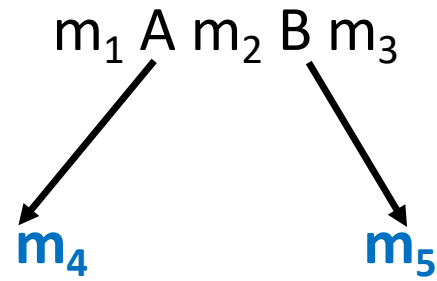
•

|

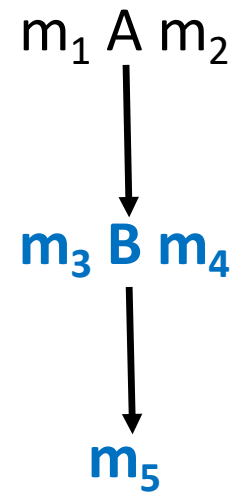
*

Arbre de dérivation

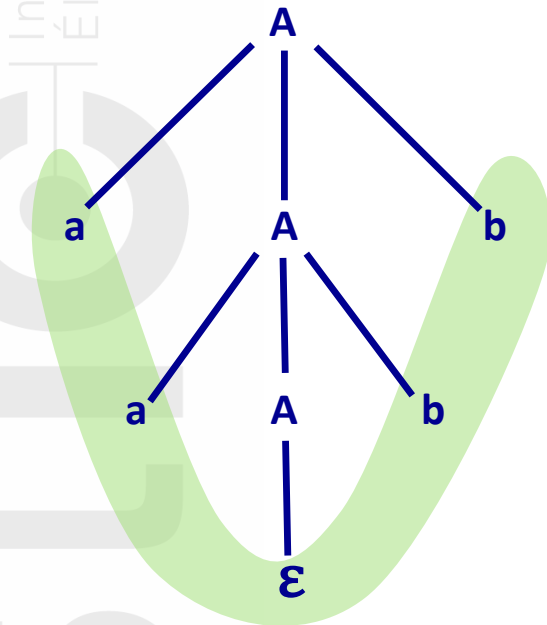
Arbre de dérivation



Arbre de dérivation



Arbre de dérivation



- Le **mot des feuilles**

$$G = \{ A \rightarrow a A b, A \rightarrow \varepsilon \}$$

$$a a b b \in \mathcal{L}(G)$$

Ambiguïté

Ambiguïté

À retenir absolument !

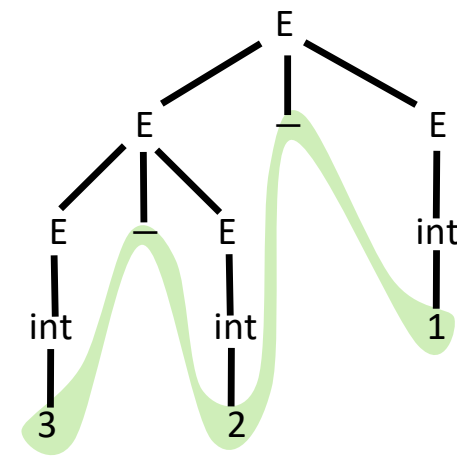
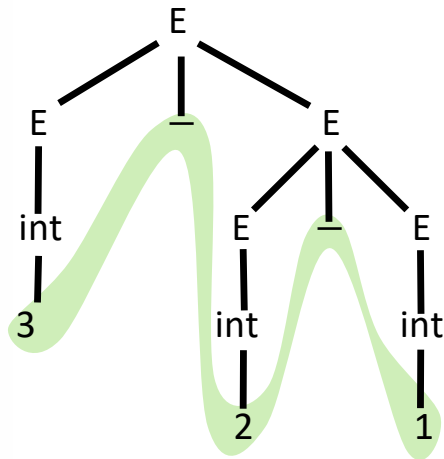
- Def. Une **grammaire** est **ambigüe**
ssi

elle produit **différents arbres de dérivation**
pour un **même mot généré**

Ambiguïté

- $G = \{ E \rightarrow E' - E, E \rightarrow \text{int} \}$

$3 - 2 - 1 \in \mathcal{L}(G)$



Ambiguïté

- On ne sait pas si il faut lire $3 - 2 - 1$
comme $(3 - 2) - 1$ ou comme $3 - (2 - 1)$
- **L'ambiguïté empêche de déterminer la sémantique**

$$\text{SEM}((3 - 2) - 1) = 0$$
$$\text{et SEM}(3 - (2 - 1)) = 2$$

Ambiguïté

- L'**ambiguïté** est une propriété d'une **grammaire...**

...si un **langage** ne peut être engendré que par des **grammaires ambiguës** on pourra dire qu'il est **ambigu**

Syntaxe et sémantique

- Le calcul de la sémantique est guidée par la syntaxe
- La conception de la syntaxe est guidée par la sémantique



Ne pas penser l'un sans l'autre

Lever l'ambiguïté

- Aucune ambiguïté ne doit être supportée...
...pas toujours possible



$\{ a^m b^n a^p b^q c \mid m=n \vee p=q \}$
est intrinsèquement ambigu

Lever l'ambiguïté

- Moyens extra-grammaticaux
 - fixer les portées des opérateurs
- Moyens grammaticaux
 - raffiner la grammaire
- Changer la syntaxe
 - souvent mal vu !

Portées des opérateurs

- Dans $\overline{1 + \overline{2 / 3}}$
 - '+' porte sur 1 et $2 / 3$ et '/' porte sur 2 et 3
 - '+' a une **plus grande portée** que '/'
 - notion plutôt syntaxique

...portée = *scope*

- Priorité $\approx$ inverse de portée
 - '/' a une **plus grande priorité** que '+'
 - notion plutôt sémantique : on évalue '/' avant '+'

Portées de opérateurs (2)

- Portées usuelles par ordre décroissant

;	<	>
:=	+	-
(ou)	*	/
&& (et)	! (non)	
== !=	** (exposant)	

- Ne résout toujours pas le problème de 3 – 2 – 1

Associativité des opérateurs

- Si $(a \text{ op } b) \text{ op } c = a \text{ op } (b \text{ op } c)$
op est **associatif** (rare !)
- Si $a \text{ op } b \text{ op } c$ doit être lu $(a \text{ op } b) \text{ op } c$
op est **associatif à gauche**
- Si $a \text{ op } b \text{ op } c$ doit être lu $a \text{ op } (b \text{ op } c)$
op est **associatif à droite**



C'est la sémantique qui décide !

Associativité des opérateurs

- Si **a op b op c** n'a pas de sens
op est **non-associatif**
- Noter le cas de **a < b < c**
il faut écrire **a < b && b < c**

Associativité usuelle des opérateurs

- **Associatif** :

(;) (+) (*)

- Associatif à **gauche** :

+ - * /

- Associatif à **droite** :

** (exposant)

= (affectation de C)

Associativité de +

- + est mathématiquement **associatif**...

...mais on le considère comme **associatif à gauche**
pour fixer l'ordre d'évaluation

Associativité de +

- + est évidemment associatif avec les **naturels...**
...mais pas avec les **flottants**

$(f + f) + F$ peut être différent de $f + (f + F)$

Rappel : $f + F = F$ si f/F trop petit



Raffiner la grammaire

- Idée : traduire la portée des opérateurs dans la structure de la grammaire

Raffiner la grammaire

- Portée de '-' > portée de '*' : $1 - 2 * 3$
- **Associativité** de '-' à gauche : $3 - 2 - 1$

expression $\rightarrow$ différence

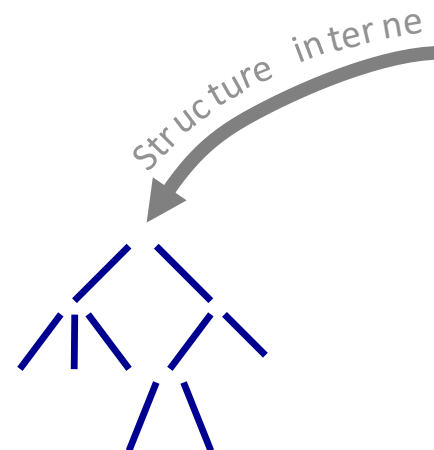
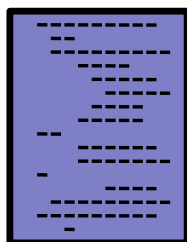
différence $\rightarrow$ différence '-' produit | produit

produit $\rightarrow$ ident | number | '(' expression ')'

Analyse syntaxique

Analyse syntaxique

grammaire +



Grammaires algébriques et automates à pile

- Th. **Un langage est engendré par une grammaire algébrique ssi il est reconnu par un automate à pile**

Preuve : $G \leftrightarrow M$

transformation de **M en G**,
et de **G en M**

- Le sens $G \rightarrow M$ est très utilisé en pratique

Analyse LL(1)

- Configuration = (**pile** $\in (\Sigma_T \cup \Sigma_{NT})^*$, $m \in \Sigma_T^*$)
 - **configuration initiale** = (S, m)
 - **configuration finale** = $(\varepsilon, \varepsilon)$
- Transitions
 - $(P \bullet a, a \bullet m) \rightarrow^a (P, m)$
 - $(P \bullet A, m) \rightarrow^\varepsilon (P \bullet \text{miroir}(\gamma_i), m)$ tq $A \rightarrow \gamma_i$
- Acceptation ssi $(S, m) \rightarrow^+ (\varepsilon, \varepsilon)$



Analyse LL(1)

- Chercher dans $(P \bullet A, m)$ des indices pour choisir $A \rightarrow \gamma_i$

**LL(1) = Left-to-right scanning,
Leftmost reduction,
1 look-ahead**

Table de prédiction

- La table de prédiction **T** permet de décider quelle règle expanser

$$T[A, a] = A \rightarrow \gamma_i$$

ssi

A n'engendre pas ϵ

et $A \rightarrow \gamma_i$ est la seule règle pour **A** qui engendre un **a** en première position

ou

A peut être suivi par un **a**

et $A \rightarrow \gamma_i$ est la seule règle pour **A** qui engendre ϵ

Table de prédiction

- Si chaque case de T contient une règle, la grammaire est LL(1)



**propriété de la grammaire,
pas du langage**

...mais si un langage n'a pas de grammaire LL(1),
on dit qu'il n'est pas LL(1)

Analyse LL(1)

pile := **S** ; $m := \text{lire}(\text{source}) \bullet \$$

tant que $m \neq \$$ faire

soit $a.m' := m$ et $\sigma := \text{dépiler}(\text{pile})$

si $\sigma \in \Sigma_T$ alors si $\sigma = a$ alors $m := m'$ lecture
sinon erreur

si $\sigma = B \in \Sigma_{NT}$ alors

si $B \rightarrow \gamma_i = T[B, a]$ alors $\text{empiler}(\text{miroir}(\gamma_i))$

sinon erreur

fsi

fait

expansion

Analyse LL(k)

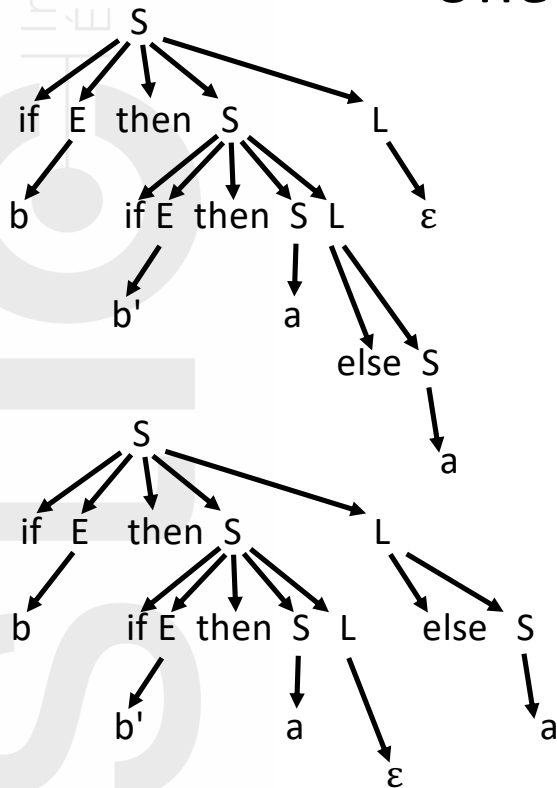
- Tout se généralise à k symboles de lecture en avant (*look-ahead*)
- JavaCC/ANTLR/Xtext fait LL(k)
- ANTLR/Xtext fait même LL(*)
 - Scala aussi !

Pragmatique de l'analyse LL(x)

- Autrefois, LL était toujours LL(1)
et LL($k > 1$) paraissait infaisable
 - d'autres stratégies étaient préférées
LALR, ex. YACC
- Maintenant, des outils LL(k) efficaces
 - **JavaCC** ou **ANTLR (Xtext)**
 - même LL(*) où k n'est pas fixé à l'avance

Pragmatique de l'analyse LL(x) (2)

- Une grammaire d'un langage qui n'est pas LL(1)



$S \rightarrow \text{if } E \text{ then } S L \mid a$

$L \rightarrow \text{else } S \mid \varepsilon$

$E \rightarrow b$

if b then if b' then a else a'



Pragmatique de l'analyse LL(x)

- déclaration $\rightarrow$ prototype | définition

prototype $\rightarrow$ type (params) ;

définition $\rightarrow$ type (params) { body }

k ?

- déclaration $\rightarrow$ type (params) suite_décl

suite_décl $\rightarrow$; | { body }



sémantique !

Pragmatique de l'analyse LL(x)

- Si lecture de k symboles suffit,
alors **G est LL(k)**
- Si G est LL(k) et engendre $\mathcal{L}$,
alors **$\mathcal{L}$ est LL(k)**
- Si G engendre $\mathcal{L}$ et n'est pas LL(k)
alors... **rien**

**Rechercher si c'est la faute du langage
ou de la grammaire**

Pragmatique de l'analyse LL(x)

- Une règle $B \rightarrow B\gamma$ est **récursive à gauche**
- $\beta \bullet B \xrightarrow{\varepsilon} \beta \bullet \text{miroir}(B\gamma) = \beta \bullet \text{miroir}(\gamma) \bullet B$
 - le sommet de pile n'a pas changé (= B)
 - le mot à lire non plus (transition sur ε)



donc, **boucle infinie**

Fuir la récursivité à gauche

- Forme normale de Greibach

– exemple :

$A \rightarrow A \alpha \mid \omega$ engendre $\omega\alpha^*$
mais est récursive à gauche

$A \rightarrow \omega A'$
 $A' \rightarrow \alpha A' \mid \varepsilon$

engendre aussi $\omega\alpha^*$
et n'est pas récursive à gauche

Fuir la récursivité à gauche

- Application aux expressions

différence $\rightarrow$ différence '-' produit | produit

engendre **produit ('-' produit)***

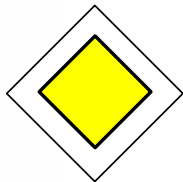
mais est récursive à gauche



différence $\rightarrow$ produit différence'
différence' $\rightarrow$ '-' produit différence' | ϵ

engendre aussi **produit ('-' produit)***

et n'est pas récursive à gauche



Fuir la récursivité à gauche

- Avantage EBNF

différence $\rightarrow$ **produit ('-' produit)***

n'est pas récursive à gauche

différence $\rightarrow$ **(produit '-')* produit**

n'est pas récursive à gauche

- La notation EBNF élimine
de nombreuses récursivités « logistiques »

Générateurs d'analyseurs syntaxiques

Principe

- Traduit une grammaire en un analyseur syntaxique
 - **grammaire $\rightarrow$ programme**
 - descente récursive
 - JavaCC ou ANTLR / Xtext
 - **grammaire $\rightarrow$ transitions + automate (= programme)**
 - YACC

Descente récursive

- Traduire une règle en une fonction

$A \rightarrow a B c \mid D e F$

```
A(m) {  
  alt = look-ahead-A(m) ; // alt  $\approx$  T[A, m.first]  
  switch alt {  
    case 1:  
      m = scan(a, m) ; m = B(m) ; return scan(c, m) ;  
    case 2:  
      m = D(m) ; m = scan(e, m) ; return F(m) ;  
  }
```

Les grammaires en résumé

- Engendrer la **structure** des mots du langages...
...en donnant des **noms** aux composants structurels
- On retiendra surtout les **grammaires algébriques**
- **Hiérarchie** cohérente de grammaires et d'automates



Les grammaires en résumé

- **Formalisme « naturel »**
- **Automatisation de $m \in \mathcal{L}(G)$ difficile en général**
- Il faut trouver des **formes particulières** qui soient automatisables **facilement**
 - **LL(1)**, **LL(k)**, **LL(*)**, **LR**, **LALR**, ...

Conclusion

- RE et grammaires algébriques métalangages de choix pour décrire des langages formels
- Très **outillées**
 - formalisation du métalangage
 - si possible préférer variantes **étendues**
- À utiliser sans modération
 - y compris **informellement**



Conclusion

- Garder la sémantique en tête
 - **fuir les ambiguïtés**
- La syntaxe **préfigure** la sémantique

Les grammaires en résumé

